

基础类型 & 字节

类型	字节	范围/备注
bool	1	true/false 注意是有符号
char	1	-128~127 注意是有符号
unsigned char	1	0~255
short	2	-32768~32767
int	4	-2 ³¹ 到 2 ³¹ -1 约 21 亿
long long	8	-2 ⁶³ 到 2 ⁶³ -1
float	4	7 位有效数字
double	8	15 位有效数字
size_t	8	0 到 2 ⁶⁴ -1, 无符号

▲ 脑残易错：表达式合法性（期中高频错题）

以下哪些声明/语句合法？为什么？

```
# int &r; → X 引用必须初始化
const int c; → X const 必须初始化
static int s; → X 静态初始化必须为 0
Student s1; → X 声明前必须非创建对象。用 Student s; 或 Student s(args);
class Foo { int x; }; → X 类定义时缺少 ;
int a[5]; a[5] = 0; → X 越界！UB，合法索引 0~4
int *p; *p = 5; → X 野指针！指向任意内存
a[i] = i++; → X UB，赋值与自增未定义顺序
int *p = new int[5]; delete p; → X 顺序为 delete[] p;，不匹配是 UB
int x = (a > b) ? a : b; → ✓ 三目运算符
int a = 1, b = (a++, a!); → ✓ 逗号表达式 b=2
(a & 5) → ✓（语法合法但逻辑通常错误，赋值返回 5→true）
int &r = 5; → X 不能绑定字面量，const int &r = 5; ✓
int* p = &a; int q = *p; → ✓ p 指向 a，解引用得 a 的值
const int* p = &a; *p = 10; → X 指向 const 不能修改
int* const p = &a; *p = &b; → X p 本身 const 不能重新绑定
char s[] = "hello"; s[0] = 'H'; → ✓ s 是字符数组可修改
char s = "hello"; s[0] = 'H'; → UB，s 是字符串常量是 const
int a = 5; int b = a++; → ✓ b=5, a=6
int a = 5; int b = ++a; → ✓ b=6, a=6
int a = 5/0; → 编译不报错，运行时报错（除零）
Base* p = new Derived; *p->f(); → ✓ f 定义多态 OK；若 f 非虚→调 Base::f()
Base b = *p; b.f(); → X 对虚对象！丢失派生数据
delete[] new int[5]; → ✓ 配对正确
for (int i = {1,2,3}) cout<<i; → ✓ C++11 initializer_list
int a[3]; a = {1,2,3}; → X 数组不能整体赋值
int f(int a){return a;} double f(double a){return a;} → ✓ 合法重载
int f(int a){return a;} double f(int a){return a;} → X 仅返回值不同
short s = 32768; → overflow, short 范围 -32768~32767
```

运算/隐式转换陷阱

- 5 / 2 = 2（整数截断），5.0 / 2 = 2.5（隐式转 double）
- 7 % 3 = -1（C++11 向零截断），不同编译器可能不同
- char c = 200; → 溢出！char 范围 128~127，用 unsigned char
- int a = 1000000; 1000000; → int 范围 -21 亿，超了
- 浮点用 eps 比较：fabs(a-b) < 1e-9，不能直接用 ==
- unsigned i; for(i=10; i>=0; i--) → **死循环**（无符号永>0）
- bool b = -1; → true（非零即真）
- int a = 3.14; a==3（隐式截断，可能 warning）
- sizeof(string) 是固定大小（内部指针指向堆），不等于 strlen

输入输出

```
cin >> a >> b; cout << x << endl;
scanf("%d %f %s %c", &n, &d, str, &c); // 注意 &
printf("%2lf %02d %8.3lf\n", d, n, d);
加速：ios::sync_with_stdio(false); cin.tie(nullptr);（之后勿混用 scanf/printf）argv
for(int i=0; i<n; i++) scanf("%d", &a[i]); // 之后勿混用 scanf/printf）argv
陷阱：>> 留在缓冲区，getline 读到空行，用
cin.ignore(numeric_limits<streamsize>::max(), '\n'); 清除。cin 读到 EOF 返回 false，
可能 while((c = cin.get()) != 'x')。
```

关键语法坑

宏定义：#define SOR(x) ((x)*(x))，不双括号 SOR(1+2)+1+2+5+9 头文件保护！
#if 0...#endif 或 #pragma once #include <file> 搜系统路径。
#include "file" 先找当前目录再搜系统路径，编译时定义：extern int x; 声明（可多次），int x; 定义（只能一次）。默认参数从右往左：void f(int a, int b=0, int c=1);
调用 f(1, f(1,2)) 函数重载：参数类型/个数/顺序不同，仅返回值不同不算重载！

指针 & 引用

```
int a = 10;
int *p = &a; // 指针，存地址
int &r = a; // 引用，a 的别名
ref = 20; // a = 20
*p = 30; // a = 30
引用 vs 指针：
引用：可写，不可空，不可多，不可变。
指针：可写，可空，可多，可变。
const 引用可绑定临时对象：const string& s。指针运算：p+1 移动 sizeof(*p) 字节。
*(arr+1) = arr[1]。数组退化：数组传参退化为指针，丢失长度。函数内数组(a)是指针，大小可变，不能取地址。
void (int a)[8]; // sizeof(a)=8，不是 8*sizeof(int)=32
野指针：delete 后未置 nullptr；返回局部变量地址（悬空指针）。空指针：C++11 用
nullptr，替代 NULL 和 0。
```

内存管理

```
int *p = new int(5); // 一个 int 初始化为 5
int *arr = new int[100]; // 100 个 int
delete[] arr; // 必须配对！不匹配是 UB。delete 两次是 UB。malloc/free 不调构造/析构，C++ 优先用 new/delete。
```

位运算

```
& | ^ ~ <<>>
int bit = (n >> k) & 1; n |= (1 << k); n &= ~(1 << k); n ^= n;
```

字符串

```
#include <string>
string s; strcpy(d,s); strcat(d,s); strcpy(s1,s2); // 0 相等 <0 s1<s2 >0 s1>s2
string s = "hello"; s.length(); s.size(); s == "world";
s.substr(0,3); s.find("ll"); // 返回位置，空 string：npos
s.c_str(); stoi("42"); stod("3.14"); to_string(42); getline(cin,s);
终止符：string 内部多存 '\0'，strlen 不到 n。c_str() 指针在修改 string 后可能失效！
```

生存期 & 作用域

范围	变量	示例
自动进入，离开域离开作用域	局部变量	int main(){...}
静态，程序启动，程序结束	全局/静态局部	static int x;
静态，new，delete，int 不 new 进	静态	static int x;

生命周期：生存期：作用域-编译期名字可见范围。生存期-运行时对象存在时间。static 局部变量：只初始化一次，跨调用保持值（作用域仍在函数内）。全局变量：在所有函数外定义，程序全程存在。滥用导致难以理解和测试。RAII：构造获取资源，析构释放资源。vector 用 RAII 管理内存。

编译 & Make & 分离编译

```
g++ -c main.cpp -o main.o # 编译 -o
g++ -o main main.o # 链接
g++ -std=c++17 -Wall -o main main.cpp
all: main
main: main.o stu.o
g++ -o main main.o stu.o
clean:
rm -f main.o stu.o
分离编译：a 放声明，.cpp 放定义。ClassName::func 作用域解析。头文件改变→引用该头文件的所有源文件全部需重编译。单个 .cpp 改→只重编译该 .cpp。
```

STL：常用容器

```
#include <vector>
vector<int> v = {1,2,3}; v.push_back(4); v.pop_back();
v.size(); v.empty(); v.clear(); v.front(); v.back();
v[1]; // 不检查越界 v.at(1); // 检查越界抛出 out_of_range
v.reserve(n); // 预分配

#include <map>
map<string,int> m; m["key"]=42; m.count("key"); m.find("key");

#include <unordered_map> // 哈希表 O(1)平均
unordered_map<int,int> um;

#include <set>
set<int> st; st.insert(3); st.count(3); st.erase(3);

#include <queue>
queue<int> q; q.push(1); q.front(); q.pop();
priority_queue<int> pq; // 大顶堆
priority_queue<int,vector<int>,greater<>> pq2; // 小顶堆
pq.top(); pq.pop();

#include <stack>
stack<int> sk; sk.push(1); sk.top(); sk.pop();

#include <deque>
deque<int> dq; dq.push_front(1); dq.push_back(2);

#include <utility>
pair<int,int> p = {1,2}; p.first; p.second;
```

STL：算法

```
#include <algorithm>
sort(v.begin(), v.end());
sort(v.begin(), v.end(), greater<>()); // 降序
nth_element(v.begin(), v.begin()+k, v.end()); // 第k小
swap(a,b); min(a,b); max(a,b);
find(v.begin(), v.end(), val); // 返回() 若未找到
count(v.begin(), v.end(), val);
reverse(v.begin(), v.end()); // 去重(先sort)重新求尾
lower_bound(v.begin(), v.end(), val); // sva 第一个
upper_bound(v.begin(), v.end(), val); // sva 第一个
binary_search(v.begin(), v.end(), val); // bool
find_if(v.begin(), v.end(), pred); // 必须检查!end()!
#include <numeric>
accumulate(v.begin(), v.end(), 0); // 求和
自定义比较：bool cmp(const T&a, const T&b){return a.x<b.x;} cmp 必须满足严格弱序：
若 cmp(a,b)=true 则 cmp(b,a)=false，lambda 表达式：sort(v.begin(),v.end(),[])
```

排序算法对比

	最好	平均	最坏	复杂度	适用
冒泡	O(n)	O(n ²)	O(n ²)	是	数据量小
选择	O(n ²)	O(n ²)	O(n ²)	是	数据有序
插入	O(n)	O(n ²)	O(n ²)	是	数据有序
堆排	O(nlogn)	O(nlogn)	O(nlogn)	否	数据无序
快排	O(nlogn)	O(nlogn)	O(nlogn)	否	数据无序
归并	O(nlogn)	O(nlogn)	O(nlogn)	否	数据无序
稳定快排	O(nlogn)	O(nlogn)	O(nlogn)	否	数据无序

算法特性：相等元素不改变相对顺序。插入/堆排不稳定。选择/快排/归并不稳定。冒泡提前终止。一趟无交换(swapped=false)则已有序（最优 O(n)）。选择排序交换次数最多 n-1 次，比冒泡少。插入排序适合有序子数组数据。std::sort 内部 introspective sort（快排+堆排+插入 sort），保证 O(n log n) 最坏。Big-O 直觉：n=10 时 O(n²)=100 次，O(n log n) 33 次差不多；n=1000 时差 15 倍；n=10000 时差 750 倍。

std::vector 要点

size()=已用元素数，capacity()=已分配空间。push_back 触发重新分配→之前所有指针引用用迭代器失效！reserve(n) 预分配避免多次重分配。[] 不检查越界(UB)，at() 检查引用超出 of range。shrink to fit() 请求释放多余容量。2D 矩阵：vector<vector<char>> board(BOARD_SIZE, vector<char>(BOARD_SIZE, ' ')); board[i][j];

迭代器与区间：算法操作 [begin, end)，左开右闭；end 是最后一个元素的下一个位置。for(vector<int>::iterator it=v.begin(); it!=v.end(); ++it) cout<<*it; for(auto it=v.begin(); it!=v.end(); ++it) cout<<*it; // auto 简化 for(const auto& x: v) cout<<x; // range 用于遍历 vector<int> v; range(v.begin(), v.end()) 返回迭代器范围。迭代器失效：push_back/insert/erase 可能导致迭代器失效，之后不可再用。

防御性编程

访问检查：角标，安全。边界：v[1] UB，v.at(1) 越界异常。*it 不检查 if(it!=v.end())。堆排 pop_back() 空 if(!v.empty())。文件：不检查打开 if(!ifstream)。野指针：用 try/catch，坐标索引用 isValidSquare 验证。delete 后置 nullptr。

文件 I/O

```
#include <fstream>
ofstream of("f.txt"); // 写，默认截断
if(!of.is_open()) cout<<"f.txt 不存在";
ifstream if("f.txt"); // 读
if(!if.is_open()) cout<<"f.txt 不存在";
getline(in, line); // 读整行含空格
ofstream bin("f.bin", ios::binary);
bin.write(reinterpret_cast<const char*>(val), sizeof(val));
ifstream f("f.bin", ios::ate; ios::binary);
long long sz = f.tellg(); // 文件字节大小
文 vs 进制：文本 4-2 字节/字符，二进制 4-8 字节/字节。序列化：顺序必须一致！不能 write64，sizeof(vec)/vec[] (vector 有内部指针)。版本标记：out<<"\n"; 第一行不能写 magic，ios::ate 定位末尾，ios::trunc 截断，用 | 组合。
```

MVC 架构

Model：数据+规则，不依赖图形界面。View：渲染，只读 const&。Controller：处理输入调 Model，View 不改 Model，Model 不碰图形，Controller 不画图。

结构体 & 枚举 & 命名空间 & 异常

```
struct Point{double x,y;}; // 默认 public
union Data{int i;double d;}; // 共享内存
enum Color{RED, GREEN, BLUE};
enum class Dir{UP,DOWN,LEFT,RIGHT}; // 强类型，必须 Dir::UP
namespace NS{int f1();}
using namespace NS; // 慎用
try{throw runtime_error("err");}
catch(const out_of_range&){e.what();}
catch(const exception&){}
catch(...){} // 不是异常干啥构造函数
```

杂项

return *this：链式调用。sizeof：编译期，类含对齐补齐。mutable：const 函数中可修改。智能指针：weak_ptr，shared_ptr，weak_ptr。p=make_shared<T>(args); 引用计数。arrow->等价 (*ptr)。explicit 禁隐式构造。constexpr 编译期常量。

Lambda 表达式

```
[[capture](params)->ret body]
auto f = []{int x}{return x*x;}; // 无捕获
auto g = [&]{int x}{return x*x;}; // 引用捕获全部
auto h = [=]{int x}{return x*x;}; // 值捕获全部
auto k = [&n]{int x}{return x*n*x;}; // 混合捕获
auto l = [](){mutable static int i; return i++;};
[i] 值捕获 → lambda 内存变量可拷贝，不可修改（除非 mutable）。[&] 引用捕获 → 注意悬垂引用（lambda 活得比变量久→UB）。泛型 lambda (C++14)：auto f=[&](auto a,auto b){return a+b};
```

constexpr 与 const

const：运行期不可修改，值可在运行期确定。constexpr：编译期常量，必须编译期可求值。const int x = foo(); // 仅当 foo() 是 constexpr 时才 OK。constexpr int y = foo(); // 编译期常量大小必须常量。constexpr int arr[10]; // 编译期数组大小必须常量。constexpr 函数可在编译期执行（如参数是编译期常量）。C++14 起 constexpr 函数可含循环和局部变量，if constexpr (C++17)：编译期条件分支。

RAII（资源获取即初始化）

构造函数获取资源，析构函数释放资源——异常安全的基础。class FileGuard{ FILE* f; public: FileGuard(const char* p){f=fopen(p,"r"); ~FileGuard(){if(f)fclose(f);} // 自动释放}; void process(){ FileGuard fg("data.txt"); // 异常或return都安全 // ... 不用手动fclose}; 标准库：lock_guard, unique_lock, ifstream 等全是 RAII。

迭代器失效 & erase

vector：push_back 扩容→全部失效；insert/erase→插入点之后失效。list/map/set insert 不失效，erase 仅失效被删元素迭代器。正确写法：it = v.erase(it);（返回下一个有效迭代器）。

OOP：类 & 构造/析构

```
class Obj {
public: int x; string name;
Obj():x(0),name(""){} // 默认构造
Obj(int a, string n):x(a),name(n){} // 带参
Obj(const Obj& o):x(o.x),name(o.name){} // 拷贝
~Obj(){} // 析构，无返回值
};
构造调用：
代码块：调用
Obj a(x); // 静态构造
Obj b=a; // 拷贝构造
func(a); // 值传递
return a; // 拷贝构造
a=b; // 拷贝赋值 operator
初始化列表必须用于：const 成员、引用成员、无默认构造的成员对象、基类初始化。初
始化顺序：静态成员变量→静态常量成员（静态类未构造完成，vptr 指向当前类的 vtable）
class A{ int b; A& operator=(const A&& a){ // b 先初始化！此时 a 未初始化→UB
explicit 禁隐式转换。=default 保留默认，=delete 删除。对象数组需默认构造。
```

▲ 脑残易错：构造 & 析构

- id=id; → 参数遮蔽效应，用 this->id=id 或初始化列表。
- 构造中调用函数→多态（派生类未构造完成，vptr 指向当前类的 vtable）
- 析构不能抛异常，大多数无返回值不可重载。
- Obj() 创建临时对象非声明变量！Obj p; 才是声明。
- 数组元素析构按顺序（最后一个是先析构）。
- 基类析构函数必须是 virtual 的，否则 delete basePtr 不调派生析构。

static & const 成员

```
class A{
public: static int cnt; // 类共享，类外定义
static int id; // 每对象一份不可改
static const int M=50; // 整型类内初始化
};
int A::cnt=0; // 必须类外定义！最易忘
static 成员函数：无 this，只能访问 static 成员。const 成员函数：void print() const; 不修改成员，mutable：const 函数中仍可修改。const 3 位置：const int* 指针，int* const 常指，const int& 常非常。
```

封装 & 访问控制

	同类	派生类	外部
private	✓	✓	✗
protected	✓	✓	✗
public	✓	✓	✓

Getter/Setter 模式：私有数据+公有接口，setter 可验证。struct 默认 public，class 默认 private。

继承

```
class Derived:public Base{ // public 继承"是 a"
public: Derived(int a):Base(a){} // 显式调用基类构造
};
构造顺序：基类→成员对象（声明时）→派生类体。析构顺序：LIFO 派生体→成员（反序）→基类。三种派生类型：public protected private
public 继承：pub pub pro inacc
protected 继承：pru prp pro inacc
private 继承：pri pri pro inacc
多重继承：class C:public A,public B{}; 菱形→虚继承 class B:virtual public A{}; 最底层类直接初始化虚基类 D():(A(x),B(x),C(x)){ is-a 继承空三角 is-a 组合空心菱形 菱形含空心菱形。UML：+pub -pri #pro。
```

虚函数 & 多态

```
class Base{public: virtual void f(){} virtual ~Base(){};}; // 基类析构 virtual!
class Derived:public Base{public: void f() override{};};
virtual ~Derived(){}; // 派生类析构 virtual!
delete p; // 虚析构函数 Derived
纯虚：virtual void f(){}→抽象类不可实例化。override：编译器检查。final：阻止重载。final void f(){}→静态绑定(编译期)→看指针类型。动态绑定(运行期)→看实际对象。只有析构引用才能多态！Base b=0;→对象切片（派生数据丢失，多态失效）。dynamic cast 需要虚函数，失败返 nullptr(指针)或抛异常(引用)。
```

拷贝控制（三法则/五法则）

```
class PalString{ char* text;
public: PalString(const char*s){text=new char[strlen(s)+1]; strcpy(text,s); PalString(const PalString&o){ // 深拷贝构造 text=new char[strlen(o.text)+1]; strcpy(text,o.text); PalString operator=(const PalString&o){ // 深拷贝赋值 if(this!=&o){delete[] text; text=new char[strlen(o.text)+1]; strcpy(text,o.text); return *this; } PalString(&o){delete[] text;};
};
三法则：需析构/拷贝构造/拷贝赋值之一→三个都需要。自赋值检查 this!=&o 不可忘！
```

friend & 运算符重载

```
class A{friend void show(A&); friend class B; private: int s;};
// 友元不能继承，不继承，不重载。Vec operator=(const Vec&& const); // 双目成员 Vecc operator=(); // 前置++& 调用 Vec operator++(int); // 前置++ 调用 friend ostream& operator<<(ostream&,const Vec&); // <<必须全局/友元
模板 & 类型转换
template<typename T> T mymax(T a,T b){return a>b?a:b;};
template<typename T> int M=class Stack{T data[N];}; Stack<int,100> s;
template<> void swap(MyType& MyType&a,MyType&b); // 特化 static const int(d); dynamic const int(d); const Cast&int(d); const Cast&int(d); reinterpret_cast<int>(p);
模板偏特化实例化，定义放头文件。隐式推导：mymax(3,5)→T=int; 显式：mymax<double>(3.5,0)。
```

完整的构造/析构顺序示例

```
class Trace{string label; public: Trace(string s):label(s){cout<<"Creating: "<<label<<endl; ~Trace(){cout<<"Destroying "<<label<<endl;};}; void nested(){ // 输出： Trace a[outer]; // Creating: outer Trace b[in2]; // Creating: in2 Trace c[in2]; // Creating: in2 ~Trace d[end]; // Destroying: in2 ~Trace d[end]; // Destroying: in2 ~Trace c[in2]; // Destroying: end ~Trace b[in2]; // Destroying: outer};
```

继承链多态完整示例

```
class Person{protected: int id; string name; public: Person(int i, string n):id(i),name(n){}; virtual void print()const{cout<<"Name:"<<name<<endl;};}; class Student:public Person{ double gpa; public: Student(int i, string n, double g):Person(i,n,g){g()}; void print()const override{cout<<"Student:"<<name<<" gpa:"<<gpa<<endl;};}; class Professor:public Person{ string dept; public: Professor(int i, string n, string d):Person(i,n),dept(d){}; void print()const override{cout<<"Prof:"<<name<<" dept:"<<dept<<endl;};}; int main(){ vector<Person*> people = { new Person(1,"Alice"), new Student(2,"Bob",3.5), new Professor(3,"Dr.Smith","CS")}; for(Person* p:people){p->print(); delete p;}; // 动态绑定}; // 输出： Person:Alice / Student:Bob gpa:3.5 / Prof:Dr.Smith dept:CS
```

抽象类 & pure virtual

纯虚函数=0 纯类或抽象类，不可实例化。纯虚函数可有实现（Base::f()），子类仍需 override。纯虚析构必须提供实现（即使=0），否则链接错误。

override & final

override 让编译器验证确实覆盖了基类虚函数（防手误签名不匹配）。final 阻止虚函数被进一步覆盖，或阻止基类继承：class A final{}

